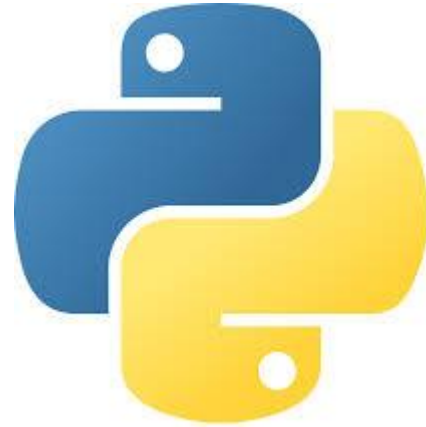




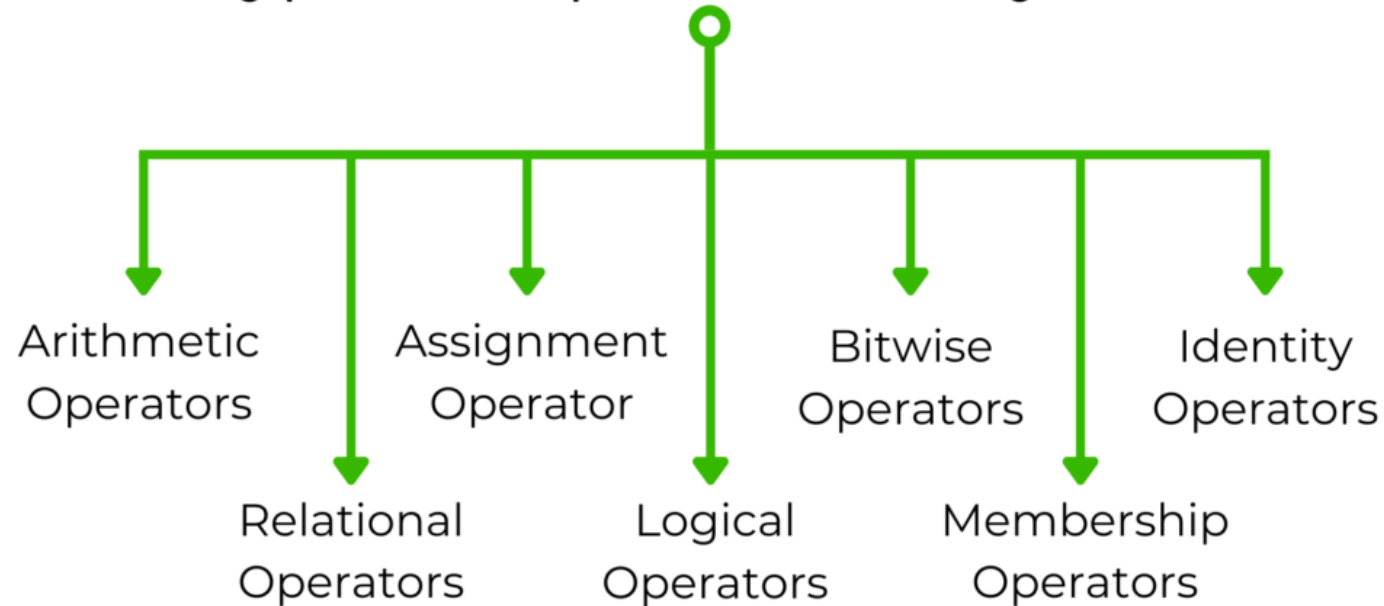
Python Programming

Shaik Abdul Gaffar, MCA
Business & Management Department
Yogi Vemana University

Python Operators

- In Python programming, Operators in general are used to perform operations on values and variables.
- **Operators:** Special symbols like -, + , * , /, etc.
- **Operands:** Value on which the operator is applied.

Types of Operators in Python



Arithmetic Operators

- Python operators are fundamental for performing mathematical calculations
- Arithmetic operators are used to perform operations on numerical values
- Common arithmetic operations include:

Operator	Description	Syntax
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division (Float)	x / y
//	Division (Floor)	$x // y$
%	Modulus	$x \% y$
**	Exponentiation (Power)	$x ** y$

Addition Operator (+)

- Used to add two values

Example:

```
val1 = 2  
val2 = 3  
  
# using the addition operator  
res = val1 + val2  
print(res)
```

Output:

5

Subtraction Operator (-)

- Used to subtract the second value from the first value

Example:

```
val1 = 2  
val2 = 3  
  
# using the subtraction operator  
res = val1 - val2  
print(res)
```

Output:

-1

Multiplication Operator (*)

- Used to find the product of two values

Example:

```
val1 = 2
val2 = 3

# using the multiplication operator
res = val1 * val2
print(res)
```

Output:

```
6
```

Float Division (/)

- Always returns a float value
- Even if both operands are integers

Example:

```
print(5/5)
print(10/2)
print(-10/2)
print(20.0/2)
```

Output:

```
1.0
5.0
-5.0
10.0
```

Modulus Operator (%)

- Used to find the remainder after division

Example:

```
val1 = 3
val2 = 2

# using the modulus operator
res = val1 % val2
print(res)
```

Output:

```
1
```

Floor Division (//)

- Returns the floor value of the quotient
- If any operand is float, result is float
- If result is negative, value is rounded down

Example:

```
print(10//3)
print(-5//2)
print(5.0//2)
print(-5.0//2)
```

Output:

```
3
-3
2.0
-3.0
```

Exponentiation Operator (**)

- Used to raise a number to the power of another

Example:

```
val1 = 2
val2 = 3

# using the exponentiation operator
res = val1 ** val2
print(res)
```

Output:

```
8
```

Precedence of Arithmetic Operators

- Operator precedence decides the order of evaluation

Operator	Description	Associativity
**	Exponentiation	Right to Left
%, *, /, //	Modulus, Multiplication, Division, Floor Division	Left to Right
+, -	Addition, Subtraction	Left to Right

Example of Operator Precedence

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Output:

```
14
20
```

Assignment Operators

- Used to assign values to variables
- Some assignment operators also perform calculations
- To write shorter and cleaner code

Operator	Example	Meaning
=	a = 5	Assign value 5 to variable a
+=	a += 3	a = a + 3
-=	a -= 2	a = a - 2
*=	a *= 2	a = a * 2
/=	a /= 2	a = a / 2

Simple Assignment Operator (=)

- Assigns a value to a variable

Example:

```
a = 5  
print(a)
```

Output:

```
5
```

Add and Assign Operator (+=)

- Adds a value and assigns the result

Example:

```
a = 5  
a += 3  
print(a)
```

Output:

```
8
```

Subtract and Assign Operator (-=)

- Subtracts a value and assigns the result

Example:

```
a = 5  
a -= 2  
print(a)
```

Output:

3

Multiply and Assign Operator (*=)

- Multiplies a value and assigns the result

Example:

```
a = 5  
a *= 2  
print(a)
```

Output:

10

Divide and Assign Operator (/=) Modulus Assignment Operator (%=)

- Divides a value and assigns the result
- Result is always a float

Example:

```
a = 10  
a /= 2  
print(a)
```

Output:

```
5.0
```

- The Modulus Assignment Operator is used to take the modulus

Example:

```
a = 3  
b = 5  
a %= b  
print(a)
```

Output:

```
3
```


Relational Operators

- Relational operators, also called Comparison Operators) are used to compare two values
- They return a Boolean result: **True** or **False**
- Python supports comparison between:
 - Numbers
 - Strings
 - Booleans
- String comparison is based on lexicographic (alphabetical) order

Operator	Name	Example
<code>==</code>	Equality	<code>a == b</code>
<code>!=</code>	Inequality	<code>a != b</code>
<code>></code>	Greater Than	<code>a > b</code>
<code><</code>	Less Than	<code>a < b</code>
<code>>=</code>	Greater Than or Equal To	<code>a >= b</code>
<code><=</code>	Less Than or Equal To	<code>a <= b</code>

Equality Operator (==)

- Checks whether two values are exactly the same
- Returns **True** if values are equal, otherwise **False**

Example:

```
a = 9
b = 5
c = 9

print(a == b)
print(a == c)
```

Output:

```
False
True
```

Inequality Operator (!=)

- Checks whether two values are **not equal**
- Returns **True** if values are different

Example:

```
a = 9
b = 5
c = 9

print(a != b)
print(a != c)
```

Output:

```
True
False
```

Greater Than Operator (>)

- Checks if the left operand is greater than the right operand

Example:

```
a = 9  
b = 5  
  
print(a > b)  
print(b > a)
```

Output:

```
True  
False
```

Less Than Operator (<)

- Checks if the left operand is smaller than the right operand

Example:

```
a = 9  
b = 5  
  
print(a < b)  
print(b < a)
```

Output:

```
False  
True
```

Greater Than or Equal To Operator ($\geq$)

- Checks if the left operand is greater than or equal to the right operand

Example:

```
a = 9
b = 5
c = 9

print(a >= b)
print(a >= c)
print(b >= a)
```

Output:

```
True
True
False
```

Less Than or Equal To Operator ($\leq$)

- Checks if the left operand is less than or equal to the right operand

Example:

```
a = 9
b = 5
c = 9

print(a <= b)
print(a <= c)
print(b <= a)
```

Output:

```
False
True
True
```

Chaining Comparison Operators

- Python allows multiple comparisons in a single statement
- Makes conditions more readable and compact

Example:

```
a = 5

print(1 < a < 10)
print(10 > a <= 9)
print(5 != a > 4)
print(a < 10 < a*10 == 50)
```

```
True
True
False
True
```

Advantage

Instead of writing:

```
1 < a and a < 10
```

You can simply write:

```
1 < a < 10
```

- Cleaner syntax
- Better readability
- More Pythonic way of writing conditions

Logical Operators in Python

Logical operators combine conditional statements. They work with Boolean values: True and False. The output is always True or False.

Operator	Meaning	Example
and	Returns True if both conditions are True	<code>a > 5 and a < 10</code>
or	Returns True if at least one condition is True	<code>a > 5 or a < 3</code>
not	Reverses the result	<code>not(a > 5)</code>

Logical AND Operator (and)

Returns True only if both conditions are True

Truth Table:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Example of AND Operator

```
a = 7  
print(a > 5 and a < 10)  
print(a > 10 and a < 20)
```

Output:

```
True  
False
```


Logical OR Operator (or)

Returns True if any one condition is True

Truth Table:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Example of OR Operator

```
a = 7  
print(a > 5 or a < 3)  
print(a > 10 or a < 3)
```

Output:

```
True  
False
```


Logical NOT Operator (not)

Reverses the Boolean result. True becomes False, False becomes True.

Truth Table:

A	not A
True	False
False	True

Example of NOT Operator

```
a = 7  
print(not(a > 5))  
print(not(a < 5))
```

Output:

```
False  
True
```

Logical Operators with Comparison Operators

Logical operators are often used with comparison operators

Example:

```
age = 18  
print(age >= 18 and age <= 25)
```

Output:

```
True
```

Important Points to Remember

Logical operators work with Boolean values

- Logical operators work with Boolean values
- and needs all conditions to be True
- or needs at least one condition to be True
- not reverses the result

Summary

- Logical operators combine conditions
- Python has three logical operators: and, or, not
- Widely used in decision-making statements

Python Bitwise Operators

Bitwise operators perform operations at bit level on integers only. Numbers are first converted into binary form, operations are applied on each bit, and the final result is converted back to decimal format.

Note: Bitwise operators work only on integers. They do not work on float or string values.

Operator	Description	Syntax
&	Bitwise AND	<code>x & y</code>
	Bitwise OR	<code>x y</code>
~	Bitwise NOT	<code>~x</code>
^	Bitwise XOR	<code>x ^ y</code>
>>	Bitwise right shift	<code>x >></code>
<<	Bitwise left shift	<code>x <<</code>

Decimal Number	Binary Number	Decimal Number	Binary Number
1	1	11	1011
2	10	12	1100
3	11	13	1101
4	100	14	1110
5	101	15	1111
6	110	16	10000
7	111	17	10001
8	1000	18	10010
9	1001	19	10011
10	1010	20	10100

Bitwise AND Operator (&)

- Compares bits of two numbers
- Result bit is 1 only if both bits are 1
- Otherwise result bit is 0

Bitwise AND Manual Example

$$X = 7 = (111)_2$$
$$Y = 4 = (100)_2$$

$$\begin{array}{r} 111 \\ \& 100 \\ \hline 100 \rightarrow 4 \end{array}$$

Bitwise AND Code Example

```
a = 10  
b = 4  
print(a & b)
```

Output:

0

Bitwise OR Operator (|)

- Compares bits of two numbers
- Result bit is 0 only if both bits are 0
- Otherwise result bit is 1

Bitwise OR Manual Example

$$X = 7 = (111)_2$$

$$Y = 4 = (100)_2$$

$$\begin{array}{r} 111 \\ | 100 \\ \hline 111 \end{array} \rightarrow 7$$

Bitwise OR Code Example

```
a = 10  
b = 4  
print(a | b)
```

Output:

14

Bitwise XOR Operator (^)

- XOR means Exclusive OR
- Result bit is 1 if bits are different
- Result bit is 0 if bits are same

Bitwise XOR Manual Example

$$X = 7 = (111)_2$$

$$Y = 4 = (100)_2$$

$$\begin{array}{r} 111 \\ ^ 100 \\ \hline 011 \rightarrow 3 \end{array}$$

Bitwise XOR Code Example

```
a = 10  
b = 4  
print(a ^ b)
```

Output:

14

Bitwise NOT Operator (~)

- Unary operator (uses only one operand)
- Converts 1 to 0 and 0 to 1
- Returns one's complement
- Python uses 2's complement, so result is negative

Bitwise NOT Manual Example

$X = 5 = (101)_2$
 $\sim 101 = 010$

Bitwise NOT Code Example

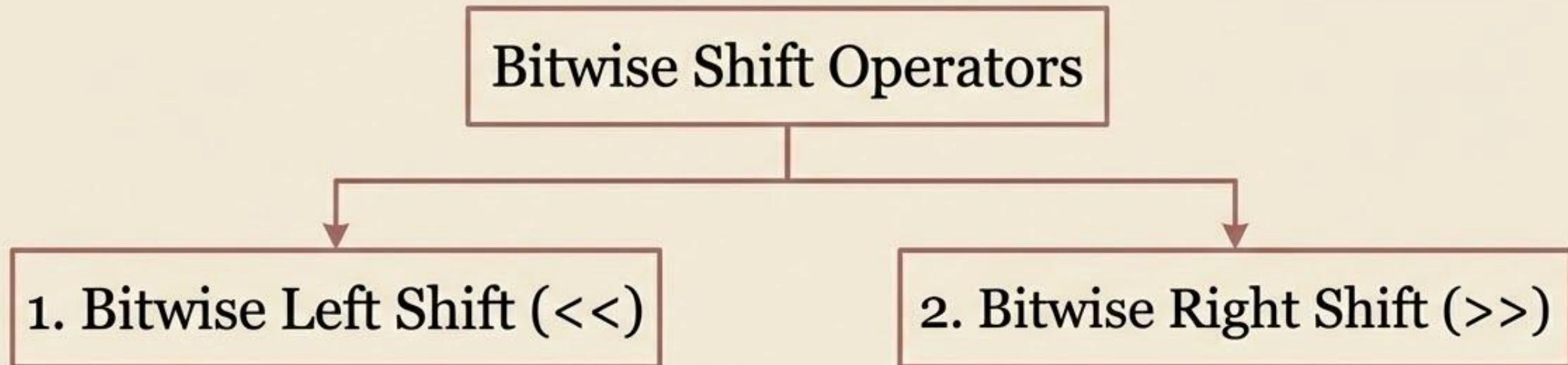
```
a = 10  
print(~a)
```

Output:

-11

Bitwise Shift Operators

- Used to shift bits left or right
- Help in multiplying or dividing by powers of 2
- These are two types:



Bitwise Left Shift Operator (<<)

The left shift operator (<<) shifts the bits of a number to the left side by a given number of positions.

Syntax: $x \ll n$

- x = number
- n = number of positions to shift

Important Rule

When you shift left:

Bits move to the left. Zeros (0) are added on the right side.

It is equal to multiplying the number by 2^n .

$$x \ll n = x \times 2^n$$

Bitwise Left Shift (<<): Examples

Example 1: $5 \ll 1$

- Step 1: Convert 5 to binary
 $5 = 00000101$
- Step 2: Shift left by 1 position
Before: 00000101
After : 00001010
- Step 3: Convert back to decimal
 $00001010 = 10$

Final Answer: $5 \ll 1 = 10$

Because: $5 * 2^1 = 5 * 2 = 10$

Example 2: $7 \ll 2$

- Step 1: Convert 7 to binary
 $7 = 00000111$
- Step 2: Shift left by 2 positions
Before: 00000111
After : 00011100
- Step 3: Convert back to decimal
 $00011100 = 28$

Final Answer: $7 \ll 2 = 28$

Because: $7 * 2^2 = 7 * 4 = 28$

Bitwise Right Shift Operator (>>)

The right shift operator (>>) shifts the bits of a number to the right side by a given number of positions.

Syntax: $x \gg n$

- x = number
- n = number of positions to shift

What Happens in Right Shift?

- Bits move to the right
- The rightmost bits are removed
- Zeros (for positive numbers) are added on the left
- It is equal to integer division by 2^n .

$x \gg n = x \div 2^n$ (For positive numbers)

Bitwise Right Shift (>>): Examples

Example 1: $8 \gg 1$

- Step 1: Convert 8 to binary
 $8 = 00001000$
 - Step 2: Shift right by 1 position
Before: 00001000
After : 00000100
 - Step 3: Convert back to decimal
 $00000100 = 4$
- Final Answer: $8 \gg 1 = 4$
Because: $8 / (2^{**} 1) = 8/2 = 4$

Example 2: $20 \gg 2$

- Step 1: Convert 20 to binary
 $20 = 00010100$
 - Step 2: Shift right by 2 positions
Before: 00010100
After : 00000101
 - Step 3: Convert back to decimal
 $00000101 = 5$
- Final Answer: $20 \gg 2 = 5$
Because: $20 / (2^{**} 2) = 20/4 = 5$

Bitwise Right Shift (>>): Negative Numbers in Python

Sign Extension

- For negative numbers, Python uses sign extension.
- The leftmost bit (sign bit) is 1 for negative numbers.
- When shifting right, Python fills the left side with 1s, not 0s.
- So the number remains negative.

Mathematical Behavior

- Right shift behaves like floor division by 2^n :

$$x \gg n = \text{floor}(x \div 2^n)$$

 **Important:** For negative numbers, Python does floor division, not normal division.

Membership Operators in Python

Membership operators are used to check whether a value exists inside a sequence.

There are 2 membership operators:

Operator	Meaning
<code>in</code>	Returns True if value is present
<code>not in</code>	Returns True if value is NOT present

They work with:

- Strings
- Lists
- Tuples
- Sets
- Dictionaries (checks keys)

Membership Operator Examples (in / not in)

Example 1 (Using in)

```
numbers = [10, 20, 30, 40]  
print(20 in numbers)
```

Explanation:

- Python checks if 20 is present in the list
- 20 exists inside the list

Output:
True

Example 2 (Using not in)

```
text = "hello world"  
print("z" not in text)
```

Explanation:

- Python checks if "z" is NOT present in the string
- "z" is not inside "hello world"

Output:
True

Important Note

In Strings:

It checks characters or substring

```
print("he" in "hello") # True
```

Because "he" exists inside "hello"

In Dictionary:

It checks only keys, not values

```
student = {"name": "Ali", "age": 22}
```

```
print("name" in student) # True
```

```
print("Ali" in student) # False
```

Because:

- "name" is a key
- "Ali" is a value

Simple Meaning

in → "Is it inside?"

not in → "Is it NOT inside?"

Identity Operators in Python

Identity operators are used to check whether two variables refer to the same object in memory.

⚠ Important:

- They do NOT check values.
- They check memory location (object identity).

There are 2 Identity Operators:

Operator	Meaning
<code>is</code>	Returns True if both variables refer to the same object
<code>is not</code>	Returns True if both variables refer to different objects

Identity Operators in Python

Example 1 (Using is)

```
a = [1, 2, 3]
b = a
print(a is b)
```

Explanation:

- `b = a` means both point to the same list
- They share the same memory location

Output:

True

Because both `a` and `b` refer to the same object

Example 2 (Using is not)

```
x = [10, 20]
y = [10, 20]
print(x is y)
```

Explanation:

- Values look same
- But they are two different list objects
- They are stored in different memory locations

Output:

False

If we check:

```
print(x == y)
```

Output:

True

Because:

- `==` checks value
- `is` checks memory location

Identity Operators in Python

Important Difference

Operator	Checks
<code>==</code>	Values
<code>is</code>	Memory location

Simple Understanding

- `==` → "Are the values equal?"
- `is` → "Are they the same object?"

The input() Function in Python

Purpose and Behavior

- The input() function is used to take input from the user.
- It waits for the user to type using the keyboard.
- When Enter is pressed, the entered value is returned.

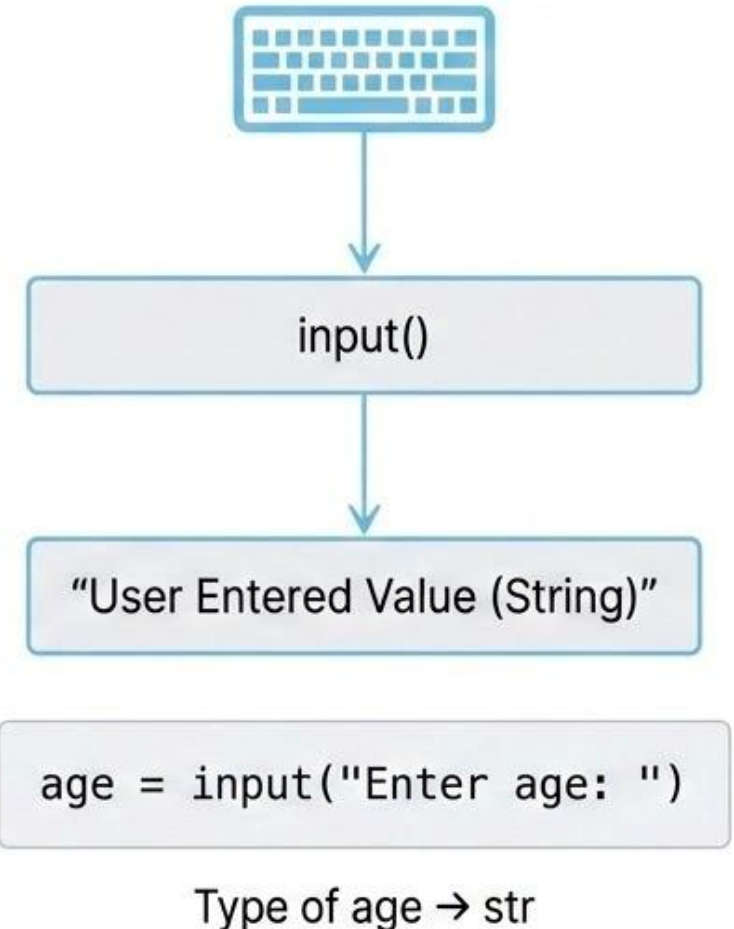
Important Point

- By default, input() always returns data as a string.
- Even if the user enters a number, it is stored as a **string**.

Syntax

```
input(prompt)
```

prompt (optional): A message displayed before taking input.



Examples of input()

Example 1: Getting a Name

```
name = input('What is your name? ')
```

User types: Emily

Then: Stored in variable: 'Emily'

Example 2: Getting an Age (Type Check)

```
age = input('Enter your age: ')\nprint(type(age))
```

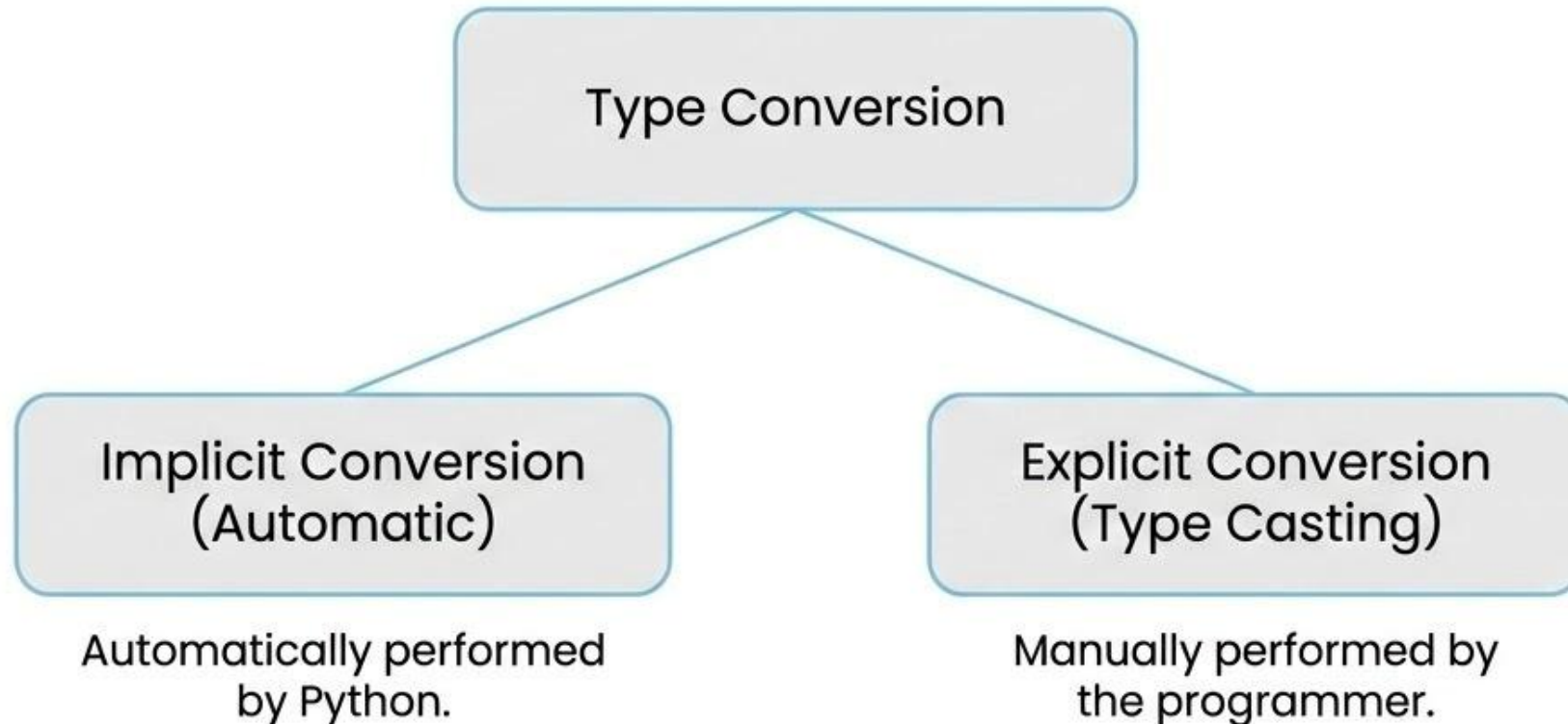
Output: <class 'str'>

Even numbers entered using **input()** are stored as strings.

input() → Always Returns → String

Type Conversion in Python

- Type conversion in Python is the process of changing a value from one data type to another.
 - Helps ensure correct operations and calculations.
 - Python supports two types of type conversion.

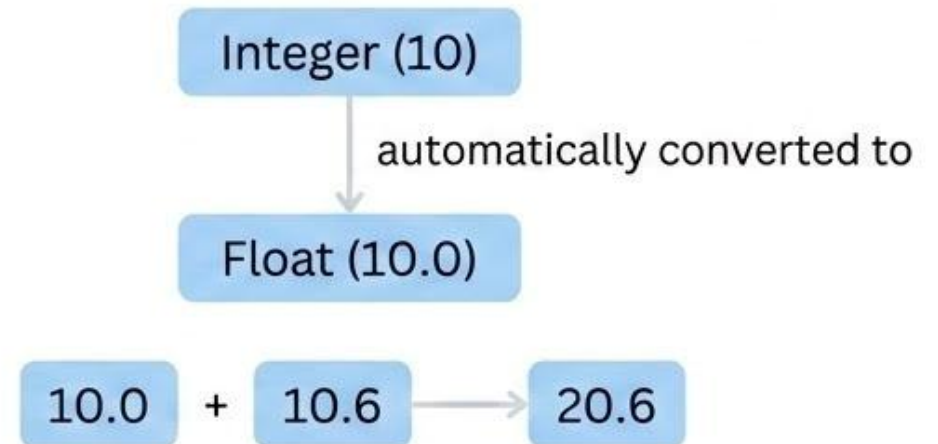


Implicit Type Conversion

- Implicit conversion in Python happens automatically when different data types are used together in an expression.
- Python converts a smaller data type to a larger one when needed.
- Commonly occurs when integers and floats are combined.
- Conversion happens at runtime to keep results accurate.

```
x = 10    # Integer  
y = 10.6  # Float  
z = x + y
```

Output: **20.6**

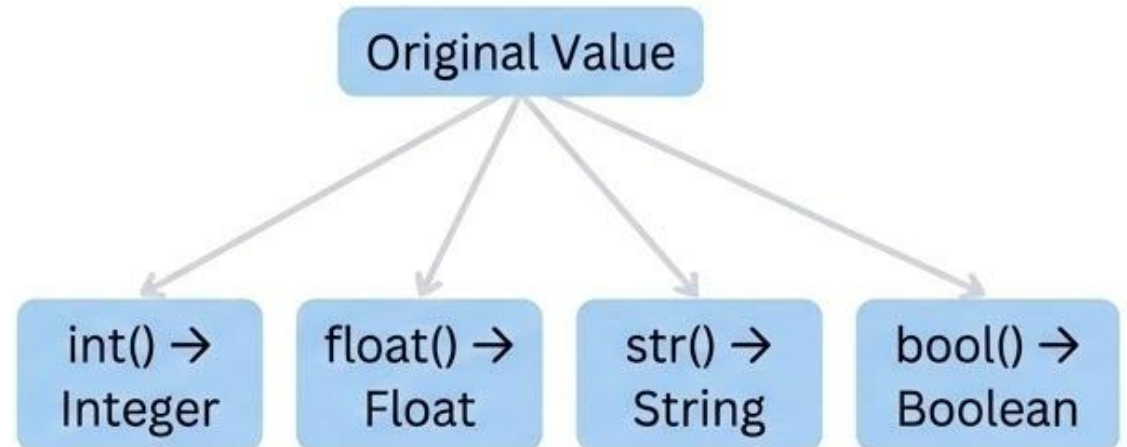


Explicit Type Conversion

- Explicit conversion, also called type casting, is when a programmer manually changes a value from one data type to another.
- Done using Python's built-in functions like `int()`, `float()`, and `str()`.
- Gives full control over how data is interpreted or processed.
- Used when automatic conversion is not suitable.

Common Type Casting Functions

- `int()` converts a value to an integer
- `float()` converts a value to a floating point number
- `str()` converts a value to a string
- `bool()` converts a value to a Boolean (True/False)



Explicit Type Conversion Functions in Python

int()

```
x = 5.8  
print(int(x))
```

Output:

5

⚠ Decimal part is removed. It does NOT round the number.

float()

```
x = 10  
print(float(x))
```

Output:

10.0

Integer becomes decimal form.

str()

```
x = 100  
print(str(x))  
print(type(str(x)))
```

Output:

"100"

<class 'str'>

Number becomes string.

Additional Type Conversions in Python

bool()

- 0 → False
 - Empty string → False
 - Empty list → False
 - All other values → True
- 0 → False
Empty → False
Value → True

```
print(bool(0))  
print(bool(5))  
print(bool(""))  
print(bool("hello"))
```

Output:

```
False  
True  
False  
True
```

complex()

```
print(complex(5))
```

Output: (5+0j)

5 → (5+0j)

j represents imaginary part.



Invalid Type Conversion

```
int("hello")
```

Error occurs because the string is not a numeric value.

Examples of Explicit Type Conversion

Example 1: Converting a Binary String

```
s = "10010"  
a = int(s, 2)  
print(a)
```

```
b = float(s)  
print(b)
```

Output

```
18  
10010.0
```

"int(s, 2)"

→ Converts binary string to decimal

"float(s)"

→ Converts string to floating point

Example 2: ASCII, Hexadecimal and Octal Conversion

```
c = '4'  
print("ASCII of '4':", ord(c))  
print("56 in Hex:", hex(56))  
print("56 in Octal:", oct(56))
```

Output

```
ASCII of '4': 52  
56 in Hex: 0x38  
56 in Octal: 0o70
```

'4' → ord() → 52

56 → hex() → 0x38

56 → oct() → 0o70

More Examples of Explicit Type Conversion

Example 3: String to Tuple, Set and List

```
s = 'geeks'
print("To tuple:", tuple(s))
print("To set:", set(s))
print("To list:", list(s))
```

Output

```
To tuple: ('g', 'e', 'e', 'k', 's')
To set: {'e', 'g', 'k', 's'}
To list: ['g', 'e', 'e', 'k', 's']
```

'geeks'

tuple() →
Ordered sequence
with duplicates

set() →
Unordered
unique elements

list() →
Ordered mutable
sequence

Example 4: Other Conversions – Complex, String, Dictionary

```
a = 1
tup = (('a', 1), ('f', 2), ('g', 3))
print("To complex:", complex(1, 2))
print("To string:", str(a))
print("To dict:", dict(tup))
```

Output

```
To complex: (1+2j)
To string: 1
To dict: {'a': 1,
'f': 2, 'g': 3}
```

1, 2 → complex() → (1+2j)

1 → str() → "1"

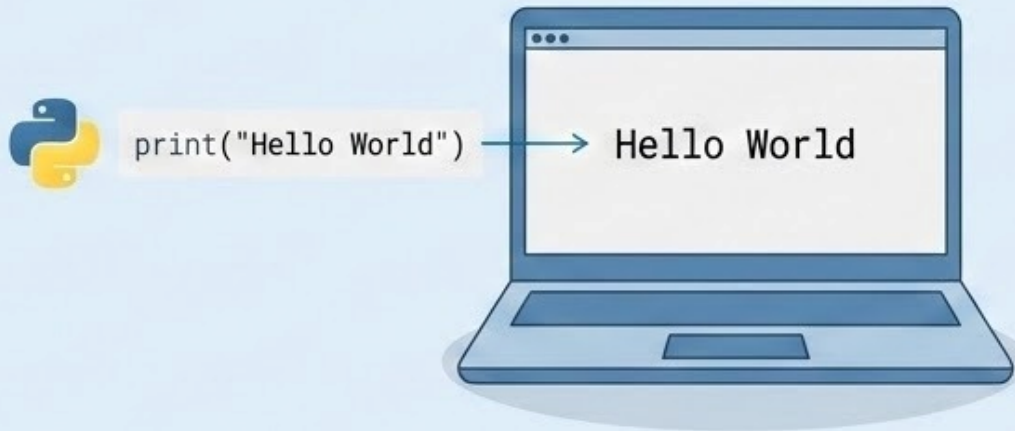
Tuple of pairs → dict() → Key-Value mapping

What is the print() Function?

"Displaying Output in Python"

The **print()** function is used to **display output on the screen**. It is a built-in function in Python, which means:

- You do not need to import anything.
- It is available by default.



```
print(object(s), sep, end, file, flush)
```

Parameter	Description
object(s)	One or more values to display
sep	Separator between multiple objects (default = " ")
end	What to print at the end (default = "\n")
file	Output destination (default = screen / sys.stdout)
flush	Forces immediate output if True (default = False)

```
print("Hello", "World")
```

↓
Hello World

Basic Printing in Python

"Understanding How the print() Function Works"

Basic Printing

```
print("Hello World")
```

Output
Hello World

The text inside quotes is displayed exactly as written.

Printing Multiple Values

```
print("Hello", "Gaffar")
```

Output
Hello Gaffar

- Python automatically adds a space between values.
- Default separator is " "

"Hello" + [space] + "Gaffar"

Printing Different Data Types

```
print(10) # int
print(5.5) # float
print(True) # boolean
print([1, 2, 3]) # list
print(("A", "B")) # tuple
```

10
5.5
True
[1, 2, 3]
('A', 'B')

The print() function automatically converts all objects into strings before displaying them.

sep Parameter (Separator)

"Controlling What Appears Between Multiple Values"

Concept Explanation

The sep parameter controls what appears between multiple values.

Default value:
sep = " " (space)

"Hello" + [space] + "World"
Default Separator

Example 1 (Date Formatting)

```
print("2026", "02", "12", sep="-")
```

Output

2026-02-12

Here, "-" is used as the separator.

Example 2 (Custom Separator)

```
print("A", "B", "C", sep=" | ")
```

Output

A | B | C

You can use any string as a separator.

Default: A B C

With sep="-": A-B-C

With sep=" | ": A | B | C

end Parameter

"Controlling What Appears at the End of a Line"

Concept Explanation

The end parameter controls what appears at the end of the output line.

Default value:
end = "\n" (new line)

```
print("Hello")  
print("World")
```

Output

Hello
World

Default new line
behavior (\n)

Default:
Hello
World

Example 1 (Replacing New Line with Space)

```
print("Hello", end=" ")  
print("World")
```

Output

Hello World

Here, the space " " replaces
the default new line.

Example 2 (Custom Ending Text)

```
print("Python", end=" --> END")
```

Output

Python --> END

You can use any string
as the ending text.

With end=" ":
Hello World

With end=" --> END":
Python --> END

Output Formatting in Python

Output formatting in Python means displaying data in a clean, readable, and user-friendly way.



Printing reports



Aligning numbers



Showing decimal precision



Combining text with variables

String Modulo Operator (%)

```
print("Age: %d" % 25)
```

format() Method

```
print("Age: {}".format(25))
```

f-Strings (Formatted String Literals)

```
age = 25  
print(f"Age: {age}")
```


String Modulo Operator (%) – Old Method

This is the traditional method (inspired by C language).

← C language
inspiration -> %

Specifier	Description	Code Example	Output
%s	String placeholder	<code>print("Hello %s" % "Gaffar")</code>	Hello Gaffar
%d	Integer (Decimal)	<code>print("Age: %d" % 23)</code>	Age: 23
%f	Float (Decimal)	<code>print("Val: %f" % 3.14)</code>	Val: 3.140000
%.2f	Float (2 Decimals)	<code>print("Price: %.2f" % 99.4567)</code>	Price: 99.46

⚠ This method is older and less **recommended** in modern Python.

format() Method – Better Way

This method is more flexible and readable.

Method Type	Code Example	Output
Basic Formatting	<pre>print("My name is {} and I am {} years old.".format("Gaffar", 23))</pre>	My name is Gaffar and I am 23 years old.
With Position (Index)	<pre>print("My name is {0} and I am {1} years old.".format("Gaffar", 23))</pre>	My name is Gaffar and I am 23 years old.
With Keywords	<pre>print("My name is {n} and I am {a} years old.".format(n="Gaffar", a=23))</pre>	My name is Gaffar and I am 23 years old.
Number Formatting	<pre>pi = 3.14159; print("Value of pi is {:.2f}".format(pi))</pre>	Value of pi is 3.14

f-Strings (Formatted String Literals) – Modern Method



Introduced in Python 3.6, this is the recommended way.

Feature	Code Example	Output
Basic Variable	<pre>name = "Gaffar"; print(f"My name is {name}")</pre>	My name is Gaffar
Multiple Variables	<pre>age = 23; print(f"I am {age} years old.")</pre>	I am 23 years old.
Decimal Formatting	<pre>pi = 3.14159; print(f"Value of pi: {pi:.2f}")</pre>	Value of pi: 3.14
Expressions / Math	<pre>a, b = 10, 20; print(f"Sum is {a + b}")</pre>	Sum is 30
Method Calls	<pre>name = "gaffar"; print(f"{name.upper()}")</pre>	GAFFAR

Most readable ✓ Most flexible ✓ Modern Python standard ✓

What is a String in Python?

Definition

A String is a **sequence of characters**. In Python, a string is **immutable**, meaning it cannot be changed directly after creation.

Example

```
name = "Gaffar"
```

The text inside quotes becomes a string.

Creating Strings Using Quotes

Single Quotes

```
s1 = 'Hello'
```

Double Quotes

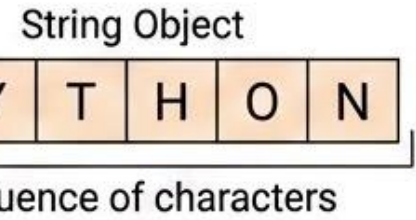
```
s2 = "Python"
```

Both single and double quotes work the same way in Python.

Triple Quotes (Multi-line String)

```
s3 = """This is\nmulti-line\nstring"""
```

Triple quotes are used for paragraphs, documentation (docstrings), or multi-line strings.



“Gaffar”

```
'Hello' = "Hello"
```

Same Output

```
Line 1  
Line 2  
Line 3
```

Multi-line Text

Important:

Python strings are immutable. Any modification creates a new string instead of changing the original one.

Special String Operations in Python

Creating an Empty String

```
s = ""
```

This creates an empty string with no characters.



Empty String
(Length = 0)

or

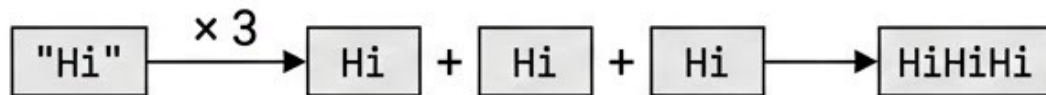
```
s = str()
```

Both methods create an empty string.

String Repetition

```
print("Hi" * 3)
```

Output: HiHiHi



Raw String

A raw string is used to **ignore escape sequences**.

```
path = r"C:\Users\Gaffar"  
print(path)
```

Normal String:

"C:\Users\Gaffar"

(May interpret \U as escape sequence)

Raw String:

r"C:\Users\Gaffar"

Treats backslashes as normal characters.



String Concatenation

```
first = "Hello"  
second = "World"  
print(first + " " + second)
```

Output: Hello World



The + operator joins strings together.

Important: Strings cannot be changed directly because they are immutable. Operations like repetition and concatenation create new strings.

What are String Methods?

String methods are **built-in functions** that are **applied to strings** to perform operations such as changing case, searching, replacing, and formatting.

Important Concept

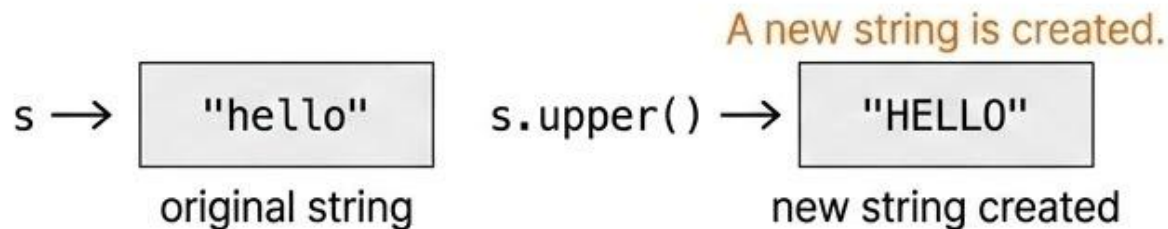
Strings are **immutable**. The **original string does not change** when a method is used.

Example

```
s = "hello"  
print(s.upper())
```

Output: HELLO

Visual Explanation (Immutability)



Proof That Original Does Not Change

```
print(s)
```

Output: hello

The variable `s` still stores the original value.

Important: All string methods return a new string because strings are immutable.



Python Built-in String Methods

<code>capitalize()</code>	<code>casefold()</code>	<code>center()</code>	<code>count()</code>
<code>encode()</code>	<code>endswith()</code>	<code>expandtabs()</code>	<code>find()</code>
<code>format()</code>	<code>format_map()</code>	<code>index()</code>	<code>isalnum()</code>
<code>isalpha()</code>	<code>isdecimal()</code>	<code>isdigit()</code>	<code>isidentifier()</code>
<code>islower()</code>	<code>isnumeric()</code>	<code>isprintable()</code>	<code>isspace()</code>
<code>istitle()</code>	<code>isupper()</code>	<code>join()</code>	<code>ljust()</code>
<code>lower()</code>	<code>lstrip()</code>	<code>maketrans()</code>	<code>partition()</code>
<code>replace()</code>	<code>rfind()</code>	<code>rindex()</code>	<code>rjust()</code>
<code>rpartition()</code>	<code>rsplit()</code>	<code>rstrip()</code>	<code>splitlines()</code>
<code>startswith()</code>	<code>strip()</code>	<code>swapcase()</code>	<code>title()</code>

Case Changing String Methods

upper()

```
s = "python"  
print(s.upper())
```

Output:

PYTHON

python

PYTHON

Converts all characters to uppercase letters.

title()

```
print("hello world".title())
```

Output:

Hello World

hello world

Hello World

Capitalizes the first letter of each word.

Method	What It Changes
<code>upper()</code>	All letters uppercase
<code>lower()</code>	All letters lowercase
<code>title()</code>	First letter of each word
<code>capitalize()</code>	First letter of whole string

lower()

```
print("PYTHON".lower())
```

Output:

python

PYTHON

python

Converts all characters to lowercase letters.

capitalize()

```
print("hello world".capitalize())
```

Output:

Hello world

hello world

Hello world

Capitalizes only the first letter of the entire string.

Important:

- These methods return a new string.
- The original string remains unchanged.

Checking Methods in Python (Return True or False)

isalpha()

```
print("Python".isalpha())
```

Output: True

Returns **True** if the string contains only **letters**.

P	y	t	h	o	n
L	L	L	L	L	L

All characters are letters.

isdigit()

```
print("123".isdigit())
```

Output: True

Returns **True** if the string contains only **digits**.

1	2	3
D	D	D

All characters are digits.

isalnum()

```
print("Python123".isalnum())
```

Output: True

Returns **True** if the string contains only **letters and numbers**.

P	y	t	h	o	n	1	2	3
L	L	L	L	L	L	D	D	D

Letters and digits allowed.

islower() and isupper()

```
print("hello".islower())
```

```
print("HELLO".isupper())
```

islower() → **True** if all letters are lowercase

isupper() → **True** if all letters are uppercase

h	e	l	l	o
---	---	---	---	---

Lowercase letters

H	E	L	L	O
---	---	---	---	---

Uppercase letters

Searching Methods in Python Strings

find()

```
s = "hello world"  
print(s.find("world"))
```

Output: 6

Returns the starting index of the substring. If not found, it returns -1.

Index: 0 1 2 3 4 5 **6** 7 8 9 10
String: h e l l o **w** o r l d
"world" starts at index 6.

If substring not found → returns -1

index()

Works the same as find().

If the substring is not found:
It raises an error instead of returning -1.

find("xyz") → -1

index("xyz") → Error

count()

```
print("banana".count("a"))
```

Output: 3

Returns the number of times a substring appears.

b **a** n **a** **n** a
 ↑ ↑ ↑

The letter "a" appears 3 times.

replace() Method in Python Strings

```
print("Hello World".replace("World",  
"Python"))
```

Hello Python

The replace() method replaces a specified substring with a new substring.

Syntax:

```
string.replace(old_value, new_value).
```

It does not modify the original string.

It returns a new string.

Step 1 – Original String:

Hello **World**

Old Value

Step 2 – After replace():

Hello **World** → Hello **Python**

New Value

Original: Hello **World**

Modified: Hello **Python**

Splitting and Joining Strings in Python

split()

```
s = "apple,banana,orange"  
print(s.split(","))
```

Output: ['apple', 'banana', 'orange']

The split() method divides a string into a list based on the separator. Original string remains unchanged.

Original String: apple,banana,orange
 ↓ ↓ ↓
Separator: ,
 ↓ ↓ ↓
Result List: ['apple', 'banana', 'orange']

join()

```
lst = ["apple", "banana", "orange"]  
print("-".join(lst))
```

Output: apple-banana-orange

The join() method combines elements of a list into a single string using the specified separator. The original list remains unchanged.

List: ['apple', 'banana', 'orange']
 ↓ ↓ ↓
Separator: "-"
 ↓ ↓ ↓
Result: apple-banana-orange

Removing Spaces in Python Strings

strip()

```
s = "    hello    "
print(s.strip())
hello
```

The strip() method removes all leading and trailing spaces from a string. The original string remains unchanged.

Original: "  hello  "

Leading spaces → removed

Trailing spaces → removed

Result: "hello"

lstrip() and rstrip()

lstrip() → Removes spaces only from the left side

rstrip() → Removes spaces only from the right side

Original:

" hello "

lstrip()
"  hello "

"hello "

rstrip()
" hello  "

" hello"

Checking String Start, End, and Length in Python

startswith()

```
s = "Python Programming"  
print(s.startswith("Python"))
```

Output: True

Returns True if the string begins with the specified substring. Otherwise, returns False.

String: "Python Programming"
Substring checked: "Python"
└─→ ✓ Match at the start → **True**

endswith()

```
print(s.endswith("ing"))
```

Output: True

Returns True if the string ends with the specified substring. Otherwise, returns False.

String: "Python Programming"
Substring checked: "ing"
└─→ ✓ Match at the end → **True**

len()

```
print(len("Python"))
```

Output: 6

Returns the total number of characters in the string, **including** letters, numbers, spaces, or symbols.

P	y	t	h	o	n
1	2	3	4	5	6

Length = 6

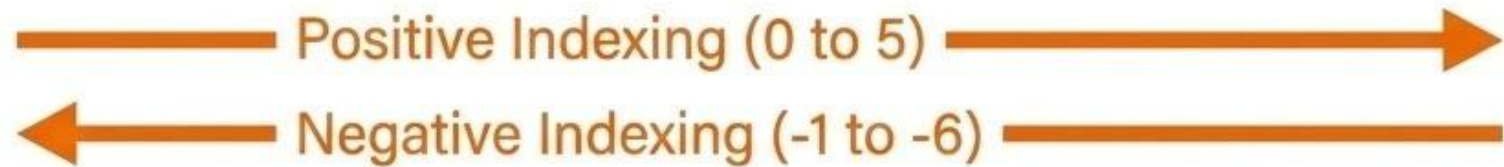
String Indexing in Python

Every character in a string has a **position number** called an **index**.
Indexing **starts from 0** in Python.

```
s = "PYTHON"
```

The string contains 6 characters.

Character	P	Y	T	H	O	N
Positive Index	0	1	2	3	4	5
Negative Index	-6	-5	-4	-3	-2	-1



Important:

Positive indexing starts from the beginning.
Negative indexing starts from the end.

Accessing Characters Using String Indexing

Given String

```
s = "PYTHON"
```

Character	P	Y	T	H	O	N
Index (+)	0	1	2	3	4	5
Index (-)	-6	-5	-4	-3	-2	-1

Positive Indexing

```
print(s[0])  
print(s[3])
```

Output:

P
H

s[0] → First character → **P**

s[3] → Fourth character → **H**

0 → 1 → 2 → 3 → 4 → 5

Left to Right Direction

Positive indexing starts from 0.

Negative Indexing

```
print(s[-1])  
print(s[-3])
```

Output:

N
H

s[-1] → Last character → **N**

s[-3] → Third character from end → **H**

-6 ← -5 ← -4 ← -3 ← -2 ← -1

Right to Left Direction

-1 = Last character

Important: Positive indexing moves from left to right.
Negative indexing moves from right to left.

String Slicing in Python

Concept

Extracting a part of a string (**substring**) is called **slicing**.

Syntax

```
string[start : end : step]
```

start → position from where slicing begins

end → position before which slicing stops

step → gap between characters (optional)

Visual Example

```
s = "PYTHON"
```

Character	P	Y	T	H	O	N
Index	0	1	2	3	4	5

Basic String Slicing

Given String

```
s = "PYTHON"
```

Character	P	Y	T	H	O	N
Index	0	1	2	3	4	5

Slicing Example

```
print(s[0:3])
```

Output:

PYT

Visual Explanation

Start index = 0

End index = 3

 (Stops before 3)

[P] [Y] [T]

The end index is not included in slicing.

Step-by-Step Breakdown

`s[0:3]`

- Start from index 0
- Move forward
- Stop before index 3
- Included indices: 0, 1, 2
- Result: PYT

Important:

- Slicing creates a new string.
- The original string remains unchanged because strings are immutable.

String Slicing Without Start or End

Given String

```
s = "PYTHON"
```

Character	P	Y	T	H	O	N
Index	0	1	2	3	4	5

Without Start Index

```
print(s[:4])
```

Output: PYTH

Since the start index is missing, Python automatically starts from index 0.

0 → 1 → 2 → 3 (Stops before 4)

P → Y → T → H

If **start is omitted**, slicing begins from **index 0**.

Without End Index

```
print(s[2:])
```

Output: THON

Since the end index is missing, Python slices until the end of the string.

2 → 3 → 4 → 5

T → H → O → N

If **end is omitted**, slicing continues until the **last character**.

Important:

- **s[:n]** means from start to index n-1.
- **s[n:]** means from index n to the end.

Full String Slice and Step Value

Given String

```
s = "PYTHON"
```

Character	P	Y	T	H	O	N
Index	0	1	2	3	4	5

Full String

```
print(s[:])
```

Output: PYTHON

Start → omitted, End → omitted

Python automatically selects the entire string.

0 → 1 → 2 → 3 → 4 → 5

`s[:]` returns a copy of the full string.

Step Value

```
print(s[0:6:2])
```

Output: PTO

Start = 0, End = 6 (not included), Step = 2

0 → 2 → 4

Selected characters: P → T → O

It selects every second character.

Step value determines the gap between selected characters.

Important:

- If step = 1 (default), every character is selected.
- If step = 2, every second character is selected.

Reverse String & Negative Slicing

Given String

```
s = "PYTHON"
```

Character	P	Y	T	H	O	N
Index (+)	0	1	2	3	4	5
Index (-)	-6	-5	-4	-3	-2	-1

Reverse String

```
print(s[::-1])
```

Output: NOHTYP

start → omitted, end → omitted, step = -1

5 ← 4 ← 3 ← 2 ← 1 ← 0

N → O → H → T → Y → P

Step = -1 moves from **right to left**, reversing the string.

Negative Slicing

```
print(s[-4:-1])
```

Output: THO

Start index = -4, End index = -1 (not included)

-4 → -3 → -2 (Stops before -1)

T → H → O

The **end index is not included**, even in negative slicing.

Important:

- **Negative indices** count from the end of the string.
- **Slicing always excludes the end index.**